## 소웨공 임시

**LOCUS: AI-Driven Household Context-Awareness for Predictive Cleaning


| | | | |
|---|---|---|---|
| Hanyeong Go<br>Hanyang University<br>Dept. of Information Systems<br>Seoul, South Korea<br>lilla9907@hanyang.ac.kr | Junhyung Kim<br>Hanyang University<br>Dept. of Information Systems<br>Seoul, South Korea<br>combe4259@hanyang.ac.kr | Dayeon Lee<br>Hanyang University<br>Dept. of Sports Science<br>Seoul, South Korea<br>ldy21@hanyang.ac.kr | Seunghwan Lee<br>Hanyang University<br>Dept. of Information Systems<br>Seoul, South Korea<br>shlee5820@hanyang.ac.kr |


Group name : LOCUS


Abstract — Current robot vacuums remain "reactive" tools, dependent on pre-set schedules or direct user commands. This limitation prevents them from effectively addressing dynamic contamination scenarios, such as "a messy kitchen immediately after cooking." To solve this, this project proposes LOCUS (Learning On-device Context and User-specific Schedules), a predictive cleaning system powered by On-device AI.

Unlike traditional cloud-based systems, LOCUS processes all data locally using a Smartphone and Laptop edge-setup, ensuring complete privacy protection. The system consists of modular components including Home Structure Intelligence (Apple RoomPlan & ArKit), Visual Context (YOLO), and Audio Context (YAMNet). A core innovation, the TimeSyncBuffer, aligns these asynchronous multi-modal streams via ZeroMQ, enabling a GRU-based sequential model to predict contamination probabilities in real-time. Plus, the system utilizes Federated Learning to further strengthen privacy protection. Ultimately, LOCUS establishes a 'Proactive Partner' system that anticipates cleaning needs based on household context rather than rigid schedules.



Edge AI, On-device AI, IoT, Artificial Intelligence, Federated Learning


Role Assignments

| Roles | Name | Task description and etc. |
|---|---|---|
| Development Manager | Seunghwan Lee | • Project Management: Oversee schedule and milestones.<br>• Mobile App Dev: Develop the spatial mapping app using Apple RoomPlan & ArKit.<br>• Audio AI: Implement the Audio Context Module using YAMNet. |
| Software Developer (AI/ML) | Hanyeong Go | • Federated Learning: Design and implement the FedPer-based PFL system.<br>• Location Intelligence: Handle coordinate data processing and synchronization logic.<br>• System Demo: Lead the final demonstration setup. |
| Software Developer (Core System) | Junhyung Kim | • Visual AI: Implement object & pose detection using YOLO.<br>• Prediction Model: Develop the GRU-based sequential prediction module.<br>• Data Fusion: Implement TimeSyncBuffer for multi-modal alignment. |
| User / Customer | Dayeon Lee | • Documentation: Lead the writing of Proposal, Specification, and Final Paper.<br>• Dashboard: Design the system monitoring dashboard (UI/UX).<br>• Planning: Define Use Cases and manage Requirements (FR/NFR). |

I. Introduction

### A. Motivation

Robot vacuum cleaners have evolved from simple cleaning tools into essential smart appliances in modern households. The recent market trend is shifting beyond powerful suction and autonomous navigation toward intelligent services that minimize user intervention. In particular, advancements in high-performance mobile processors (e.g., Apple Silicon) and robot-specific NPUs (Neural Processing Units) are enabling On-device AI environments. This technological progress allows robots to not only avoid obstacles but also perceive their surroundings in specific detail, providing the foundation for robot vacuums to evolve from passive appliances into proactive domestic partners.

### B. Problem Statement

Despite rapid hardware advancements, currently commercialized robot vacuum software still faces the following limitations (Pain Points):

· Reactive Limitations: Current robot vacuums remain "reactive" tools, dependent on pre-set schedules or direct user commands. This prevents them from effectively addressing dynamic contamination scenarios in the home, such as cleaning "the kitchen immediately after cooking" or "the dining table after a meal."

· Lack of Context Awareness: Existing systems lack the ability to judge "when, where, and why" a mess occurred. They merely avoid obstacles and fail to interpret user living patterns through visual or auditory cues, resulting in poor environmental adaptability.

· Privacy Concerns: The conventional method of transmitting camera and microphone data to cloud servers for intelligent features has led to user distrust regarding potential hacking and privacy infringements.

### C. Proposed Solution

To address these issues, this project proposes the LOCUS (Learning On-device Context and User-specific Schedules) system. LOCUS adopts an On-device AI architecture that utilizes a smartphone and laptop as edge devices to process the robot's computational tasks.

The system collects multi-modal data—including home structure (Apple RoomPlan & ARKit), visual context (YOLO), and audio context (YAMNet)—to perceive the domestic environment comprehensively. Specifically, it precisely synchronizes asynchronously collected data from different sensors using ZeroMQ communication and TimeSyncBuffer technology. This aligned time-series data is then fed into a GRU (Gated Recurrent Unit)-based sequential pattern module to predict the probability of contamination in advance.

· Predictive Cleaning based on Multi-modal Context: Instead of simple scheduling, the system implements a "proactive partner" capable of predicting when and where contamination will occur by fusing visual, auditory, and location context.

· Privacy Assurance via On-device AI: Sensitive raw data (e.g., raw images and audio) is processed exclusively within the edge device, ensuring it is never transmitted to external servers. Furthermore, the implementation of Personalized Federated Learning (FedPer) fundamentally prevents potential personal data leakage.

· User-Specific Environmental Adaptation: By learning semantic spatial labels defined via the mobile application and the user's unique living patterns, the system establishes a personalized cleaning policy optimized for each household's specific environment.

### D. Objectives & Contributions

#### 1 YOLOv11n-pose

**YOLO is a state-of-the-art, real-time object detection architecture known for its speed and accuracy. Specifically, the YOLOv11n-pose model is a lightweight version optimized for edge devices. It is employed to detect human poses and furniture within the camera frame, providing visual context about user activities.**

#### 2 YAMNet

**YAMNet is a pre-trained deep neural network developed by Google that predicts 521 audio event classes based on the AudioSet corpus. This model serves as the system's auditory sensor, classifying household sounds such as "chopping food" or "running water" to infer environmental context without recording conversations.**

#### 3 ZeroMQ

**ZeroMQ is a high-performance asynchronous messaging library aimed at use in distributed or concurrent applications. In LOCUS, ZeroMQ handles on-device communication only (IPC: `/tmp/locus_sensors.ipc`), enabling ultra-low-latency data exchange between the sensor modules (Camera, Mic, GPS) and the AI inference pipeline to ensure precise timestamp alignment in the TimeSyncBuffer.**

#### 4 MQTT (Message Queuing Telemetry Transport)

**MQTT is a lightweight publish-subscribe messaging protocol designed for IoT devices. In LOCUS, MQTT serves as the primary communication layer between the Backend server and Edge devices (cross-device, Internet-based). The system uses Mosquitto as the MQTT broker with QoS=1 to ensure reliable message delivery.**

**MQTT Topics in LOCUS:**

- **Backend → Edge**: `home/{homeId}/zones/update` - Notifies edge device when user adds/removes room labels in the app
- **Edge → Backend**: `home/{homeId}/prediction/pollution` - Publishes GRU contamination predictions
- **Edge → Backend**: `home/{homeId}/cleaning/result` - Reports cleaning completion with duration
- **Edge → Backend**: `edge/{deviceId}/status` - Device heartbeat (online/offline)

### Why Both MQTT and ZeroMQ?

- **MQTT**: Backend ↔ Edge Device communication (cross-device, network-based, QoS guarantees)
- **ZeroMQ**: On-device sensor fusion (ultra-low latency <1ms, IPC-based, no network overhead)

#### 5 TensorFlow & Keras

**TensorFlow is an end-to-end open-source platform for machine learning, and Keras is its high-level API. These frameworks are utilized to construct, train, and deploy the GRU (Gated Recurrent Unit) based sequential model, which calculates the probability of future contamination based on time-series context data.**

#### 6 React Three Fiber (R3F)

**React Three Fiber is a React renderer for Three.js, a library used to create and display 3D computer graphics in a web browser. It is used in the LOCUS dashboard to visualize the 3D home structure scanned by RoomPlan, allowing users to intuitively view and manage cleaning zones.**

## II. Requirements

### A. Functional Requirements

The major functional requirements of the system are defined as follows:

#### 1 FR1. Home Structure & Location Intelligence Module:

The system shall generate a 3D indoor structure map based on Apple RoomPlan and ARKit, and map semantic labels (e.g., "Kitchen," "Living Room") input by the user via the application. Additionally, it must transmit the robot's real-time coordinates via WebSocket and ZeroMQ to share the current Zone ID with all modules in real-time. **When users add or remove room labels, the Backend must immediately notify the Edge device via MQTT to trigger GRU Head replacement.**

- FR1.1: Generation and rendering of LiDAR-based 3D RoomPlan structures
- FR1.2: Provision of an interface for user-defined semantic label input.
- FR1.3: Broadcasting of ARKit-based real-time coordinates from WebSocket to ZeroMQ.
- FR1.4: Real-time management of Zone ID as a common context referenceable by all modules.
- **FR1.5: Dynamic Zone Synchronization via MQTT**
  - When user adds/removes a room label in the mobile app, Backend publishes MQTT message ( `home/{homeId}/zones/update` ) to notify the Edge device
  - Edge device's ZoneManager receives the update and triggers GRU Head replacement to match the new number of zones
  - Example: User adds "서재" (study room) → Backend sends 5 zones → GRU Head changes from Dense(4) to Dense(5)
  - Base Layer weights are preserved (transfer learning), only Head Layer is reinitialized

#### 2 FR2. Multimodal Context Awareness:

· FR2.1 Visual Context:

The system must detect objects (furniture) and human behaviors using an on-device YOLO model via camera input and combine this with Zone information.

· FR2.2 Audio Context:

The system must analyze microphone input using the YAMNet model to classify major domestic auditory events such as dishwashing or watching TV.

· FR2.3 Timestamp Synchronization:

The system must receive asynchronously collected visual, audio, and location data via ZeroMQ, align them within a ±100ms timestamp window, and generate a single Context Vector with missing values filled.

**· FR2.4 Multi-Modal Feature Fusion via Attention Mechanism:**

The system uses an **AttentionContextEncoder** to combine different sensor data into a single context vector. This allows the model to learn relationships between different inputs (e.g., "sofa + TV sound + sitting pose → likely eating snacks in living room").

**How it works:**

- **Inputs**: 5 types of sensor data
  - Visual: 14 object classes (e.g., person, chair, laptop)
  - Audio: 17 sound classes (e.g., speech, music, dishes)
  - Pose: Human body position (17 joints)
  - Spatial: Current zone (balcony, bedroom, kitchen, living_room)
  - Time: Time of day, day of week
- **Processing**: Attention mechanism learns which combinations are important
  - Example: "cooking sound + kitchen location + standing pose" → high kitchen contamination
- **Output**: Fixed 160-dimensional vector → Fed into GRU model
- **Benefits**:
  - Flexible: Can add/remove object classes without retraining entire model
  - Interpretable: Attention weights show which sensor combinations matter most
  - Fast: ~50ms processing time

#### 3 FR3. Sequential GRU Prediction:

The system must take the 30-timestep time-series context sequence generated by the TimeSyncBuffer as input and learn using a GRU (Gated Recurrent Unit) model. Through this, it must proactively predict the contamination probability for specific zones (e.g., under the dining table) based on current behavioral patterns.

#### 4 FR4. Personalized Federated Learning:

To protect user privacy, the model is split into a **Base Layer (Shared)** and a **Head Layer (Personalized)**. During federated learning, only the Base Layer containing common patterns is exchanged with the server, while the Head Layer containing unique habits is retained exclusively on-device.

**FedPer Architecture:**

**Base Layer (Shared across all homes):**

- Two GRU layers that learn common patterns (e.g., "kitchen gets dirty after cooking")
- Shared with server during federated learning
- ~50K parameters

**Head Layer (Personalized per home):**

- Final Dense layers that predict contamination for each zone
- Never leaves the device (privacy protection)
- ~1K parameters
- Output size adapts to number of zones (e.g., 4 zones → Dense(4), 5 zones → Dense(5))

**Dynamic Zone Adaptation:**

- When user adds/removes rooms via mobile app:
  - Backend sends MQTT message to Edge device
  - Only Head Layer is replaced to match new zone count
  - Base Layer is preserved (keeps learned behavioral patterns)
- Example: 4 zones (거실, 주방, 침실, 베란다) → User adds "서재" → Head automatically becomes Dense(5)

#### 5 FR5. Central Policy & Dashboard:

- FR5.1 Policy Execution:

The system must determine and execute policies such as "Immediate Clean," "Suggestion," or "Hold" by synthesizing contamination prediction results with the current context (e.g., Do Not Disturb mode).

- FR5.2 System Monitoring:

The system must visualize CPU/GPU usage, inference latency, and data flow between modules via a dashboard.

### B. Non-Functional Requirements

The constraints and quality requirements that the system must adhere to are as follows:

- NFR1. Privacy & Security:

User sensitive information (Raw Image/Audio) must never be transmitted outside the device, and all data processing must be performed on-device. Even during federated learning, privacy must be guaranteed by transmitting only model weights (gradients).

- NFR2. Hardware Constraints:

The system must optimize and lightweight models to enable real-time inference even in resource-constrained environments (CPU, RAM) of edge devices

- NFR3. Real-time Performance:

The processes of multi-modal data collection, synchronization, and inference must be processed without delay (Real-time) matching actual living speeds, and stable alignment of asynchronous stream data must be guaranteed.

## III. Development Environment

### A. Choice of Hardware & OS Platform

**We utilize a distributed development environment consisting of four personal high-performance laptops to simulate both the edge computing environment and the model training server. The devices in use are:**

- macOS (MacBook Air/Pro): Selected for developing iOS-specific features such as RoomPlan and ARKit. macOS provides the necessary ecosystem (Xcode) to build and test the mobile perception modules efficiently.
- Windows (Samsung Galaxy Books): Selected for training heavy deep learning models (YOLO, GRU) and running system simulations, leveraging the versatility of Windows for various IDEs and GPU compatibility.

### B. Choice of software development platform

### 1. Programming Languages

**a) Python**

**Python is a high-level, interpreted programming language known for its simplicity and extensive library ecosystem. In this project, Python serves as the core language for developing the backend server, data processing pipelines, and AI models. Its rich support for scientific computing libraries, such as TensorFlow and NumPy, enables efficient implementation of the GRU-based prediction model and complex data handling required for the On-device AI system.**

**b) TypeScript**

**TypeScript is a strongly typed superset of JavaScript that compiles to plain JavaScript. It is utilized in this project for developing the web-based dashboard and client applications. By providing static type checking, TypeScript significantly reduces runtime errors and enhances code maintainability. This ensures a robust and type-safe development environment for visualizing real-time robot data and managing user contexts.**

### 2. Frameworks & Tools

**a) Fastify**

**Fastify is a high-performance web framework for Node.js. In the LOCUS architecture, it serves as the primary backend server, handling API requests from the dashboard and mobile clients. Its low overhead and asynchronous nature make it ideal for processing real-time data streams and managing interactions with the database.**

**b) React Native**

**React Native is a UI software framework used to develop the mobile application component of LOCUS. It enables the creation of the mobile tracker that runs on the user's smartphone, collecting GPS and IMU data to assist in robot localization. It allows for maintaining a unified codebase across web and mobile platforms.**

**c) React + Vite**

**React is a JavaScript library for building user interfaces, while Vite is a modern frontend build tool that provides a fast development experience. This combination is used to build the LOCUS dashboard. React's component-based architecture allows for the creation of interactive UI elements for monitoring robot status, while Vite's Hot Module Replacement (HMR) feature ensures rapid feedback loops during development, significantly improving productivity.**

**d) ZeroMQ**

**ZeroMQ is a high-performance asynchronous messaging library. In LOCUS, ZeroMQ handles on-device communication only (IPC: `/tmp/locus_sensors.ipc` ), enabling ultra-fast communication (<1ms latency) between the sensor modules (Camera, Mic, GPS) and the AI inference pipeline. It is crucial for synchronizing multi-modal data with minimal latency using the ROS ApproximateTimeSynchronizer algorithm (±100ms window, LOCF fill strategy).**

**e) RoomPlan API**

**RoomPlan is a powerful Swift API powered by Apple, designed to create 3D floor plans of indoor spaces using the camera and LiDAR scanner on iOS devices. In the LOCUS project, RoomPlan serves as the core component for Home Structure Intelligence. It actively scans the user's physical environment in real-time, automatically recognizing and classifying structural elements such as walls, windows, doors, and furniture. This technology enables the system to generate a parametric 3D model (USDZ format) of the home, which acts as the semantic map for defining cleaning zones and context.**

**f) ARKit**

**ARKit is Apple's advanced augmented reality framework that integrates device motion tracking, camera scene capture, and advanced scene processing. It functions as the foundational layer that powers RoomPlan. In this system, ARKit utilizes Visual Inertial Odometry (VIO) to fuse sensor data from the device's accelerometer and gyroscope with visual data. This provides precise, real-time localization, allowing the LOCUS system to accurately track the position and orientation of the edge device within the generated 3D map, bridging the gap between the physical home and the digital context model.**

### 3. Infrastructure & Tools

**a) PostgreSQL**

**PostgreSQL is a powerful, open-source object-relational database system. It is employed as the central persistent storage for the LOCUS system, storing user profiles, semantic map data, and historical cleaning logs. Its robustness ensures data integrity for the entire platform.**

**b) GitHub**

**GitHub is a cloud-based platform for version control and collaboration, powered by Git. It serves as the central repository for the LOCUS project, managing the source code for the backend, frontend, and AI modules. GitHub facilitates team collaboration through features like pull requests, issue tracking, and code reviews, ensuring systematic version management and continuous integration throughout the development lifecycle.**

**c) Notion**

**Notion is an all-in-one workspace tool used for project management, documentation, and team collaboration. In this project, Notion serves as the central hub for organizing research materials, tracking development progress via Kanban boards, and documenting API specifications. Its flexibility allows the team to maintain a shared knowledge base and synchronize tasks effectively across different development phases.**

**d) Visual Studio Code**

**Visual Studio Code (VS Code) is a lightweight but powerful source code editor developed by Microsoft. It is used as the primary Integrated Development Environment (IDE) for both Python and TypeScript development. With its vast ecosystem of extensions, including support for debugging, syntax highlighting, and version control integration, VS Code streamlines the coding workflow and facilitates efficient development across the multi-modal system components.**

### 4. Programming Languages

- Hardware: $0 (Utilizing existing personal laptops: MacBook Air, MacBook Pro, Galaxy Books).
- Software: $0 (All utilized libraries and tools—Python, React, PostgreSQL, VS Code—are open-source or free for educational use). NFR3. Real-time Performance:
- Cloud Computing: $0 (All training and inference are performed locally on-device to adhere to privacy-first principles).
- Total Estimated Cost: $0

## C. Task Distribution_

| Roles | Name | Task description and etc. |
|---|---|---|
| Development Manager | Seunghwan Lee | • Project Management: Oversee schedule and milestones.<br>• Mobile App Dev: Develop the spatial mapping app using Apple RoomPlan & ArKit.<br>• Audio AI: Implement the Audio Context Module using YAMNet. |
| Software Developer (AI/ML) | Hanyeong Go | • Federated Learning: Design and implement the FedPer-based PFL system.<br>• Location Intelligence: Handle coordinate data processing and synchronization logic.<br>• System Demo: Lead the final demonstration setup. |
| Software Developer (Core System) | Junhyung Kim | • Visual AI: Implement object & pose detection using YOLO.<br>• Prediction Model: Develop the GRU-based sequential prediction module.<br>• Data Fusion: Implement TimeSyncBuffer for multi-modal alignment. |
| User / Customer | Dayeon Lee | • Documentation: Lead the writing of Proposal, Specification, and Final Paper.<br>• Dashboard: Design the system monitoring dashboard (UI/UX).<br>• Planning: Define Use Cases and manage Requirements (FR/NFR). |

# IV. Specifications

## A. A. FR1. Semantic Mapping & Localization Specification

**To achieve real-time semantic mapping and localization, the system integrates Apple's RoomPlan API with a messaging interface.**

**1. Implementation Logic:**

**· 3D Scanning: The RoomPlan API captures the environment and outputs a .usdz parametric model.**

**· Label Extraction: Semantic labels (e.g., 'Kitchen') are extracted from the CapturedRoom object and mapped to 3D coordinate bounding boxes.**

**· Coordinate Transmission: The agent's position $(x, y, z)$ derived from ARKit is broadcasted via ZeroMQ to align with the robot's local map.**

**2. Data Structure (Location Payload):**

**{**

**"timestamp": 1730612500.123,**

**"device_id": "locus_tracker_01",**

**"zone_id": "kitchen",**

**"coordinates": { "x": 2.3, "y": 1.2, "z": 1.0 }**

**}**

## B. FR2. Multimodal Context Awareness Specification

**This module utilizes a TimeSyncBuffer to align asynchronous data streams from independent sensor processes.**

**1. Sub-module Specifications:**

**· Visual Context (FR2.1): An on-device YOLOv8 model processes camera frames to detect objects (e.g., 'cup', 'sofa') and human poses. Results are published to the visual_topic via ZeroMQ.**

**· Audio Context (FR2.2): The YAMNet model analyzes 1-second audio clips to classify events (e.g., 'water_running', 'speech') and publishes probabilities to the audio_topic.**

**· Timestamp Synchronization (FR2.3): The TimeSyncBuffer subscribes to all topics and aligns data using the LOCF (Last Observation Carried Forward) strategy within a ±100ms window.**

**2. Algorithm: TimeSyncBuffer (Pseudocode):**

**Note**: This implementation adopts the **ROS ApproximateTimeSynchronizer** algorithm, which is commonly used in robotics for synchronizing multi-modal sensor streams with different sampling rates. The algorithm maintains per-sensor queues and matches messages within a configurable time window (slop).

**The system uses the Last Observation Carried Forward (LOCF) strategy to handle different sampling rates (Vision: 30fps, Audio: ~1Hz).**

```
class TimeSyncBuffer:
def process_stream(self):
    # 1. Initialize ZeroMQ Subscribers
    visual_sub = zmq.socket(SUB, "visual_topic")
    audio_sub = zmq.socket(SUB, "audio_topic")
    while True:
        # 2. Fetch Data (High Frequency Visual ~30fps)
        visual_data = visual_sub.recv_json()
        # 3. Check for Audio Data (Lower Frequency ~1Hz)
        # Non-blocking check for asynchronous alignment
        try:
audio_data=audio_sub.recv_json
(flags=zmq.NOBLOCK)
            self.last_audio = audio_data # Update state
        except zmq.Again:
```

```
        # LOCF Strategy: Carry forward last known audio state
        audio_data = self.last_audio
    # 4. Feature Concatenation & Vector Generation
    # Context Vector = [Visual(14) + Audio(17) + Location(7) + Time(10)]
    context_vector = np.concatenate([
        visual_data,
        audio_data,
        get_realtime_location(),
        get_time_encoding()
    ])
    # 5. Push to Rolling Buffer for GRU Input
    self.sequence_queue.append(context_vector)
    if len(self.sequence_queue) == 30:
        trigger_gru_prediction(self.sequence_queue)
```

**C. FR3. Sequential GRU Prediction Specification**

**The synchronized context sequence is processed by a sequential deep learning model to predict contamination risks.**

**1. Model Architecture:**

- **Input Layer: Accepts a sequence of shape (Batch_Size, 30, Feature_Dim) representing 30 timesteps of context history.**
- **Hidden Layers: Utilizes stacked GRU (Gated Recurrent Unit) layers (e.g., 64 units) to capture temporal dependencies in user behavior.**
- **Output Layer: A Dense layer with Sigmoid activation outputs the Contamination Probability for each defined zone.**

**2. Output Example:**

```
{
  "prediction_timestamp": 1730612505,
  "probabilities": {
"kitchen": 0.83,
"living_room": 0.21,
"dining_table": 0.92
  }
}
```

**D. FR4. Personalized Federated Learning Specification**

**To strictly adhere to privacy requirements, the system implements the FedPer (Federated Personalization) architecture.**

**1. Layer Splitting Logic:**

- **Layer Splitting: The GRU model is architecturally divided into a Base Layer (Feature Extractor) and a Head Layer (Classifier).**
- **Local Training: The device trains the entire model using local private data.**
- **Selective Sharing: During the aggregation phase, the system extracts only the weights of the Base Layer. The Head Layer weights, which contain user-specific habit patterns, are never serialized or transmitted to the server.**

**2. Federated Update Loop (Pseudocode):**

```
def on_federated_round_start(server_weights):
# 1. Load ONLY Base Layer weights from server
    model.base_layer.set_weights(server_weights)
# 2. Train locally on private user data
model.fit(local_dataset, epochs=5)
# 3. Upload ONLY Base Layer gradients
# Head Layer remains strictly on-device
return model.base_layer.get_gradients()
```

**5 FR5. Central Policy Specification**

- **FR5.1 Policy Execution:**

**The Policy Engine acts as a state machine. It subscribes to GRU predictions and checks the current system mode (e.g., 'Do Not Disturb'). If the predicted probability exceeds the threshold (e.g., 0.8) and the context is permissible, a "Clean" command is published via ZeroMQ.**

- **FR5.2 System Monitoring:**

**1) Tech Stack: A React-based web dashboard connected via Fastify backend.**

**2) Visualizations: Real-time graphs displaying CPU/GPU usage, model inference latency (ms), and the active flow of messages across ZeroMQ topics.**

## V. Architecture Design & Implementation (partial)

**A. Overall Architecture**

**The LOCUS is designed with a hybrid architecture that combines on-device edge computing for real-time inference with a cloud-based infrastructure for federated learning and monitoring**

**1 System Overview:**

The overall architecture connects the App Backend, IoT Edge Device (Robot), and the Federated Learning Network. The robot processes visual and auditory data locally using ZeroMQ for low-latency communication, while the cloud handles non-sensitive metadata and model weight aggregation.

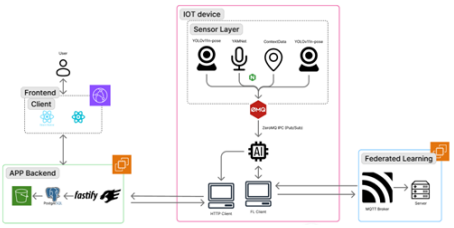


**Figure 5: Overall System Architecture & Data Flow**

## B. Directory Organization

**The project code is managed across three specialized repositories to ensure separation of concerns:**

1. **SE_G10: The main system repository containing robot control logic, AI models, and simulation environments.**
2. **LocationDataLocus: Handles mobile-based spatial mapping and location data processing.**
3. **FL: Dedicated to Federated Learning experiments and weight aggregation logic**

| Directory | Description |
|---|---|
| **SE_G10/** | Main AI & Edge inference repository |
| src/spatial_mapping/ | (FR1) GPS zone encoding and coordinate transformation |
| src/audio_recognition/ | (FR2) YAMNet-based environmental sound classification |
| src/context_fusion/ | (FR2) AttentionContextEncoder & TimeSyncBuffer for multi-modal fusion |
| src/model/ | (FR3) FedPerGRUModel implementation |
| src/dataset/ | (FR3) Dataset generators (mock sensor data, location sequences) |
| realtime/gru_predictor.py | (FR3, FR4) Real-time prediction pipeline with ZeroMQ subscriber |
| realtime/cleaning_executor.py | (FR4) LocalDecisionEngine and cleaning action executor |
| realtime/mqtt_client.py | (FR1.5, FR5) EdgeMQTTClient for Backend ↔ Edge bidirectional communication |
| realtime/zone_manager.py | (FR1.5) Dynamic zone synchronization and GRU Head replacement |
| realtime/sensor_visual.py | YOLOv11n object detection publisher (ZeroMQ) |
| realtime/sensor_audio.py | YAMNet audio classification publisher (ZeroMQ) |
| realtime/decision_engine.py | Threshold-based local decision logic |
| training/train_encoder.py | AttentionContextEncoder training script |
| training/train_gru.py | FedPerGRU training script |
| training/prepare_data.py | Dataset preprocessing pipeline |
| models/gru/gru_model.keras | Pre-trained GRU model weights |
| models/audio/yamnet.tflite | YAMNet backbone (frozen) + trained head |
| models/yolo/best.pt | YOLOv11n fine-tuned weights |
| models/attention_encoder.keras | Pre-trained AttentionContextEncoder weights |
| config/zones_config.json | Zone definitions (4 zones: balcony, bedroom, kitchen, living_room) |
| data/generated/ | Generated training datasets |
| datasets/ | Raw datasets (HD10K_yolo, homeobjects-3K, IROS2022_Dataset) |
| **FL/** | Federated Learning server & client repository |
| FL/server.py | (FR5) MQTT-based FedAvg aggregation server |
| FL/client.py | (FR5) MQTT-based FL client (simulates edge device training) |
| FL/config.py | FL configuration (zones, hyperparameters) |
| FL/dashboard/ | Real-time FL monitoring dashboard (WebSocket) |
| AI/models/gru/ | Shared GRU model architecture (same as SE_G10) |
| **LocationDataLocus/** | Mobile app & Backend repository |
| LocusBackend/ | (FR5) Backend API server (Fastify + Prisma + PostgreSQL) |
| ├─ src/modules/auth/ | User authentication (JWT) |
| ├─ src/modules/homes/ | Home CRUD operations |
| ├─ src/modules/labels/ | Zone label management (add/remove/update) |
| ├─ src/modules/mqtt/ | MQTT broker integration for zone updates |
| ├─ src/modules/users/ | User profile management |
| ├─ prisma/schema.prisma | Database schema (User, Home, Zone, Device tables) |
| LocusClient/ | (FR1) Web dashboard for home management (React) |
| ├─ src/pages/ | Home list, zone editor, cleaning history visualization |
| ├─ src/components/ | UI components (zone canvas, pollution heatmap) |
| LocusMobileTracker/server/ | (FR1) WebSocket bridge for iPhone ARKit location streaming |
| LocusTrackerExpo/ | (FR1) iOS app for RoomPlan 3D scanning + ARKit tracking (React Native + Expo) |
| ├─ expo-arkit/ | Custom ARKit native module (Swift) |
| ├─ app/(tabs)/ | UI screens (scan room, label zones, view map) |

VI. Use Cases

Use Case 1: Context-Aware Cleaning Pause ("The 'Not Now' Scenario")

1. Description This scenario demonstrates LOCUS's ability to prioritize user convenience over rigid schedules. Unlike conventional "reactive" robots that blindly adhere to preset timers, LOCUS proactively senses the user's context (e.g., attending a meeting) and intelligently defers cleaning tasks to prevent disruption.
2. Preconditions

· The robot is scheduled to clean the "Living Room" at 14:00.

· The user is currently present in the living room, engaged in a video conference.

3. Flow of Events

1. [Trigger] At 14:00, instead of immediately starting the vacuum motor, the robot activates its perception sensors.
2. [Visual Perception] The YOLO model detects the user sitting on the sofa in a static pose, facing a laptop.
3. [Auditory Perception] The YAMNet model detects continuous human speech and distinct video conference sounds (e.g., remote voices).
4. [Context Inference] The GRU model analyzes the sequence of data (Static Pose + Meeting Sounds + Living Room Location) and infers the context state as "User Busy / Meeting".
5. [Decision Making] The Policy Engine overrides the scheduled task and sets the status to "Hold".
6. [Feedback] The robot remains docked or silent and sends a quiet push notification to the user: "It seems you are in a meeting. Shall I clean the living room later?"
7. [Resumption] After 30 minutes, upon detecting the cessation of meeting sounds and the user standing up, the robot suggests: "Would you like me to clean the living room now?"

4. Impact & Differentiation

- Existing Solutions (Pain Point): A conventional robot would have started cleaning at 14:00, creating noise interference during the meeting and forcing the user to manually intervene (painful UX).
- LOCUS Solution (Value): Demonstrates "social intelligence" by understanding when not to clean, thereby transforming the robot from a simple appliance into a considerate home partner.

## A. References


[1] Apple Inc., "RoomPlan | Apple Developer Documentation," Apple Developer. [Online]. Available: https://developer.apple.com/documentation/roomplan. [Accessed: Nov. 2025].

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," version 11.0, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics.

[3] S. Hershey et al., "CNN architectures for large-scale audio classification," in Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP), New Orleans, LA, USA, 2017, pp. 131-135.

[4] P. Hintjens, ZeroMQ: Messaging for Many Applications, O'Reilly Media, 2013.

[5] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org/.

[6] Poimandres, "React Three Fiber: React renderer for Three.js," GitHub, 2024. [Online]. Available: https://github.com/pmndrs/react-three-fiber. [Accessed: Nov. 2025].

[7] L. Zhou, "Research and Simulation of A Home Robot Navigation System Based on YOLO-SBA and ROS2," in Proceedings of the 2025 2nd International Conference on Mechanics, Electronics Engineering and Automation (ICMEEA 2025), Atlantis Press, vol. 272, pp. 465–479, Aug. 2025. doi: 10.2991/978-94-6463-821-9_48.

[8] X. Han, S. Chen, Z. Fu, et al., "Multimodal Fusion and Vision-Language Models: A Survey for Robot Vision," arXiv preprint arXiv:2504.02477, Apr. 2025. To appear in Information Fusion, vol. 126, Feb. 2026.

[9] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," arXiv preprint arXiv:1912.00818, Dec. 2019.

[10] H. Yang, J. Li, M. Hao, W. Zhang, H. He, and A. K. Sangaiah, "An efficient personalized federated learning approach in heterogeneous environments: a reinforcement learning perspective," Sci. Rep., vol. 14, article 28877, 2024. doi: 10.1038/s41598-024-80048-3.

[11] M. Windl, J. Leusmann, A. Schmidt, S. S. Feger, and S. Mayer, "Privacy communication patterns for domestic robots," in Proceedings of the 2024 Symposium on Usable Privacy and Security (SOUPS 2024), USENIX, Philadelphia, PA, USA, pp. 121–138, Aug. 2024. [Online]. Available: https://www.usenix.org/system/files/soups2024-windl.pdf.

[12] TensorFlow, "Transfer learning with YAMNet for environmental sound classification," TensorFlow Documentation, 2024. [Online]. Available: https://www.tensorflow.org/tutorials/audio/transfer_learning_audio.

[13] Y. Yang, Z. Su, Y. Zhang, C. C. Goh, Y. Lin, A. G. Bellotti, and B. G. Lee, "Kolmogorov–Arnold networks-based GRU and LSTM for loan default early prediction," arXiv preprint arXiv:2507.13685, Jul. 2025.

[14] M. M. Gad, W. Gad, T. Abdelkader, and K. Naik, "Personalized smart home automation using machine learning: Predicting user activities," Sensors, vol. 25, no. 19, article 6082, Oct. 2025. doi: 10.3390/s25196082.**